

DEV_IMPL_GUIDE - SUB-LM-01 Subscription Lifecycle Management

1. Purpose

This guide explains SUB-LM implementation as runtime flows execute.

Use it with DD_SUB-LM-01-Subscription_Lifecycle_Management-v1.0.md.

SUB-LM is the subscription source of truth. Workflow modules call it; it does not orchestrate workflows.

2. Runtime Actors

Runtime component	What it does
subscription-management-api pod	Owns subscription create/read/internal mutation APIs.
subscription-management-service module	Contains state machine, cycle calculator, restriction logic.
SUB-LM repository	Owns SQL access to subscription and SUB-LM catalogs.
Kafka outbox dispatcher	Publishes SUB-LM low-level events if SUB-LM owns the outbox dispatcher.
SUB-WF workers	Call SUB-LM internal mutation APIs during BPMN execution.

If SUB-LM and SUB-WF run in one deployable app, keep modules separate in code and routes.

3. Before Requests Can Work

Required tables:

```
subscription
subscription_status_code
subscription_transition_reason
subscription_restriction
```

Required status seed values:

```
CREATED
PENDING_ACTIVATION
ACTIVE
PENDING_PAUSE
SUSPENDED
PENDING_RESUME
PENDING_UPGRADE
PENDING_DOWNGRADE
PENDING_RELOCATION
PENDING_MIGRATION
PENDING_TERMINATION
TERMINATED
RETIRED
```

Required indexes:

- Primary key on subscription_id.
- Duplicate active subscription guard on (account_id, homepass_id, package_ref) from the DD.
- Account lookup index (operator_code, account_id, status_code).
- Status/operator lookup indexes needed by APIs.

Implementation invariant:

- account_id is mandatory and is the operational anchor.
- customer_id is mandatory but denormalized for identity/search/audit convenience.
- Never decide billing, dunning, payment allocation, work order, or equipment ownership using customer_id alone.

4. Flow 1 - Create Subscription

4.1 Caller Sends Request

Endpoint:

```
POST /api/subscriptions HTTP/1.1
Authorization: Bearer <jwt-or-service-token>
Idempotency-Key: create-sub:cust_01HZ:hp_01HZ:pkg_INET
Content-Type: application/json
```

```
{
  "operatorCode": "WIK",
  "customerId": "cust_01HZ_NEW",
  "accountId": "acc_01HZ_HOME",
  "homepassId": "hp_01HZ_NRB",
  "packageRef": "pkg_INET_VOIP_RES",
  "billingMode": "PREPAID",
  "currency": "KES",
  "cycleModel": "CALENDAR_ANCHOR",
  "cycleFrequencyMonths": 1,
  "createdBy": "ful-02-service"
}
```

4.2 Request Lands On subscription-management-api

The API validates:

Field	Validation
operatorCode	Required and authorized.
customerId	Required.
accountId	Required. Must belong to customerId and operatorCode according to ILM-CFG-01.
homepassId	Required.
packageRef	Required.
billingMode	Allowed enum.
currency	Required ISO-style code.
cycleModel	Allowed cycle model.
Idempotency-Key	Required.

4.3 SUB-LM Loads Reference Data

For create, SUB-LM may validate package and HomePass by calling approved clients:

```
PackageCatalogClient -> SIP API or approved read-through cache
HomePassClient -> RLM API or approved read-through cache
```

SUB-LM business logic must not directly read Redis/cache keys.

If source API is unavailable and no valid cached snapshot exists, fail safely.

4.4 SUB-LM Writes Subscription

In one DB transaction:

1. Reserve idempotency key.
2. Check duplicate non-terminal subscription for (account_id, homepass_id, package_ref).
3. Insert subscription with status_code = CREATED.
4. Insert low-level subscription-created outbox event.
5. Store idempotency response.
6. Commit.

Important:

- Do not activate service here.
- Do not call FUL/BIL here.
- Do not start Camunda here.

4.5 API Returns Response

201 Created

```
{
  "subscriptionId": "sub_01HZ_NEW",
  "operatorCode": "WIK",
  "customerId": "cust_01HZ_NEW",
  "accountId": "acc_01HZ_HOME",
  "homepassId": "hp_01HZ_NRB",
  "packageRef": "pkg_INET_VOIP_RES",
  "statusCode": "CREATED",
  "billingMode": "PREPAID",
  "cycleModel": "CALENDAR_ANCHOR"
}
```

5. Flow 2 - Workflow Puts Subscription In Pending State

This flow is called by a BPMN worker, for example sub-lm-put-pending-status.

Endpoint or in-process service:

```
PUT /internal/subscriptions/sub_01HZ_NEW/state HTTP/1.1
Authorization: Bearer <subscription-workflow-service-token>
Content-Type: application/json
```

```
{
  "toStatus": "PENDING_ACTIVATION",
  "currentTransitionType": "ACTIVATE",
  "currentTransitionReasonCode": "SALES_FIELD",
  "operationId": "subop_01HZ_NEW_ACT"
}
```

SUB-LM executes:

1. Authenticate service token.
2. Start DB transaction.
3. Lock row:

```
SELECT *
FROM subscription
WHERE subscription_id = :subscriptionId
FOR UPDATE;
```

4. Validate transition CREATED -> PENDING_ACTIVATION.
5. Save prior status snapshot for worker response.
6. Update status and transition fields.
7. Insert outbox event.
8. Commit.

Response:

```
{
  "priorStatusSnapshot": {
    "statusCode": "CREATED",
    "activeRestrictions": []
  },
  "lmEventId": "evt_01HZ_PENDING_ACT"
}
```

6. Flow 3 - Workflow Commits Final State

Called by sub-lm-commit-state.

Request:

```
{
  "subscriptionId": "sub_01HZ_NEW",
  "toStatus": "ACTIVE",
  "expectedFromStatus": "PENDING_ACTIVATION",
  "operationId": "subop_01HZ_NEW_ACT"
}
```

SUB-LM executes:

1. Authenticate service token.

2. Start transaction.
3. Lock subscription row.
4. Verify current status equals expectedFromStatus.
5. Validate transition.
6. Set status_code = ACTIVE.
7. Clear current_transition_type and reason.
8. Set lifecycle timestamp such as activated_at if included by approved master-field flow.
9. Insert outbox event.
10. Commit.

If current status is not expected, return conflict:

```
{
  "code": "SUBSCRIPTION_STATUS_MISMATCH",
  "expected": "PENDING_ACTIVATION",
  "actual": "ACTIVE"
}
```

7. Flow 4 - Workflow Updates Master Fields

Called by sub-lm-put-master-fields.

Request:

```
{
  "subscriptionId": "sub_01HZ_NEW",
  "operationId": "subop_01HZ_NEW_ACT",
  "fieldsJson": {
    "activated_at": "2026-05-18T10:00:18Z",
    "cycle_anchor_day": 18,
    "package_version_id": "pkgver_003"
  }
}
```

SUB-LM executes:

1. Authenticate service token.
2. Lock subscription.
3. Validate only allowed fields are present.
4. Reject immutable field changes such as billing_mode.
5. Update fields.
6. Insert outbox event.
7. Commit.

Do not accept arbitrary JSON patch paths.

8. Flow 5 - Workflow Reverts To Prior Status

Called by sub-lm-revert-to-prior.

Request:

```
{
  "subscriptionId": "sub_01HZ_NEW",
  "priorStatus": "CREATED",
  "failureReasonCode": "FULFILLMENT_FAILED",
  "operationId": "subop_01HZ_NEW_ACT"
}
```

SUB-LM executes:

1. Authenticate service token.
2. Lock subscription.
3. Validate current status is a pending status.

4. Restore prior status.
5. Store last_failure.
6. Clear transition fields.
7. Insert outbox event.
8. Commit.

Response:

```
{
  "revertedAt": "2026-05-18T10:05:00Z",
  "lmEventId": "evt_01HZ_REVERT_ACT"
}
```

9. Flow 6 - Active Restrictions Are Replaced

Called by sub-lm-put-active-restrictions.

Request:

```
{
  "subscriptionId": "sub_01HZ_SUB",
  "operationId": "subop_01HZ_RESTRICT",
  "activeRestrictions": [
    {
      "code": "OUTGOING_VOICE_BARRED",
      "reasonCode": "DUNNING_LEVEL_2"
    }
  ]
}
```

SUB-LM executes:

1. Lock subscription.
2. Validate every restriction code exists.
3. Validate no duplicate code.
4. Replace full active_restrictions array.
5. Insert outbox event.
6. Commit.

Do not implement partial array patching in SUB-LM.

10. Flow 7 - Cycle Window Calculation

Called by sub-lm-compute-cycle-window.

Request:

```
{
  "subscriptionId": "sub_01HZ_NEW",
  "referenceDate": "2026-05-18"
}
```

Response:

```
{
  "cycleWindowJson": {
    "cycleStart": "2026-05-01",
    "cycleEnd": "2026-05-31",
    "daysIntoCycle": 18
  }
}
```

SUB-LM computes dates only. BIL computes money.

11. Outbox Runtime

When SUB-LM writes an event, the outbox dispatcher later executes:

```
SELECT *
FROM kafka_outbox
```

```
WHERE status = 'PENDING'  
ORDER BY created_at  
LIMIT 100  
FOR UPDATE SKIP LOCKED;
```

Then:

1. Publish to Kafka.
2. Mark PUBLISHED.
3. On failure, increment attempts.
4. Mark FAILED after retry limit.

12. Manual Smoke Test

1. Seed catalogs.
2. Create subscription.
3. Confirm status is CREATED.
4. Call internal pending-state API.
5. Confirm status is PENDING_ACTIVATION.
6. Call master-fields update.
7. Call commit-state.
8. Confirm status is ACTIVE.
9. Confirm outbox rows exist.
10. Confirm invalid transition returns conflict.

13. Common Mistakes

- Workflow logic inside SUB-LM.
- External BIL/FUL calls while holding subscription lock.
- Direct SQL from other modules against SUB-LM tables.
- Updating state without FOR UPDATE.
- Treating cache as source of truth for current subscription status.
- Treating pending states as final business facts.